Task 2: Trash Image Classification

For this task, I am using PyTorch to fine-tune a pre-trained Convolutional Neural Network (ResNet-101) to classify images of waste into six categories:

- Cardboard
- Plastic
- Glass
- Metal
- Paper
- Trash

This is following a standard Computer Vision (CV) pipeline:

- Data ingestion and augmentation (resizing, cropping, normalisation)
- Feature extraction (via the ResNet-101 convolutional layers)
- Train a classifier (fine-tuning the fully connected layer)
- Generate predictions
- Evaluate
- Error Analysis (using the confusion matrix to identify misclassifications)

I am also using Torchvision for image transformations, Matplotlib and Seaborn for data visualisation, and Scikit-learn for my evaluation metrics.

In [1]:
```
import os
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
from torchvision import datasets, models
from torch.utils.data import DataLoader, Dataset, random_split, Subset
from tqdm import tqdm
from PIL import Image
import numpy as np
```

In [2]:
```
device = torch.device("cuda") if torch.cuda.is_available() else "cpu"
print(f'Using device: {device}')
```

```
Using device: cuda
```

Custom Dataset Class and Data Ingestion

To feed our images into a PyTorch model, we need a way to convert raw image files and their text-based filenames into numerical tensors. I created a custom `TrashDataset` class that inherits from `torch.utils.data.Dataset` to handle this efficiently.

- **Dynamic Label Extraction:** Instead of relying on rigid folder structures, the `__getitem__` method parses the filename (e.g., `train_cardboard_000.jpg`) to extract the class name.
- **Label Mapping:** Neural networks output probability distributions across numbers, not text. The `label_map` dictionary acts as our lookup table, cleanly converting our string classes (like "plastic" or "metal") into integer indices (0 through 5).
- **Deterministic Loading:** Using `sorted()` on the `os.listdir` ensures that the dataset loads in exactly the same order across different machines or runs, which is crucial for reproducibility and debugging.

In [3]:
```
class TrashDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        # Keep deterministic ordering and ignore non-files (hidden files, direct
        self.file_list = sorted([f for f in os.listdir(root_dir) if os.path.isfi

        # Create a mapping for labels to numbers
        # e.g., {'cardboard': 0, 'plastic': 1}
        self.label_map = {"cardboard": 0, "plastic": 1, "glass": 2, "metal": 3,

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_name = self.file_list[idx]
        img_path = os.path.join(self.root_dir, img_name)

        # Load image
        image = Image.open(img_path).convert("RGB")

        # Extract label from filename: train_cardboard_000.jpg -> cardboard
        parts = img_name.split('_')
        if len(parts) > 1:
            label_str = parts[1]
        else:
            # fallback in case filename doesn't match expected pattern
            label_str = 'trash'
        label = self.label_map.get(label_str, self.label_map['trash'])

        if self.transform:
            image = self.transform(image)

        return image, label


# Load the base dataset (pass None for transform initially)
train_dataset = TrashDataset(root_dir='data/train', transform=None)
test_dataset = TrashDataset(root_dir='data/test',transform=None)
```

Exploratory Data Analysis (EDA)

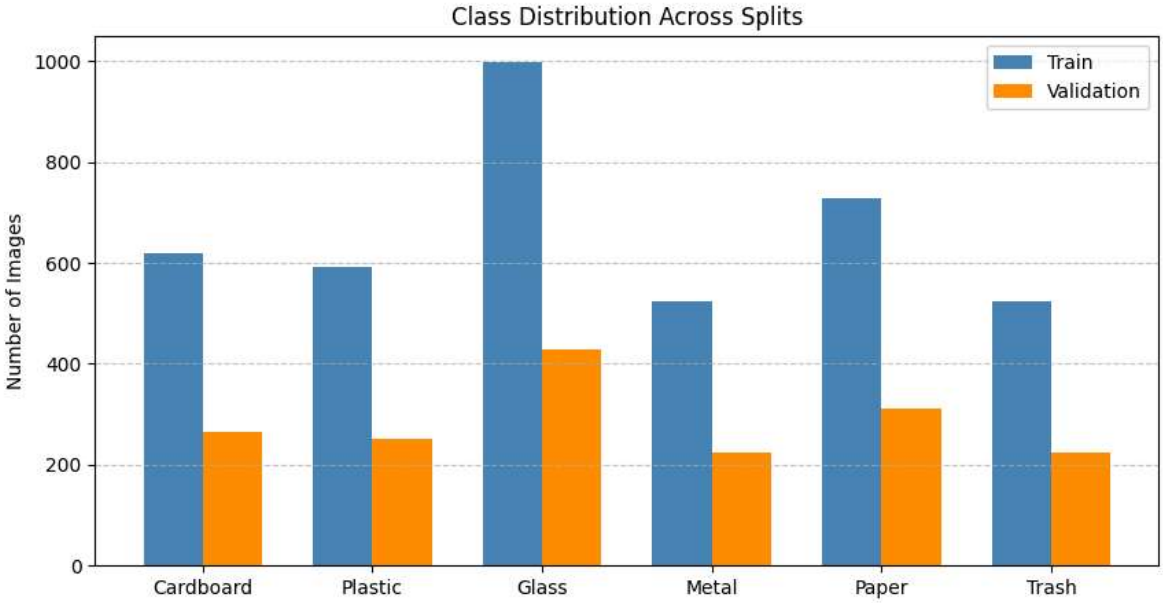
Before training any model, it is critical to understand the data we are working with. This Exploratory Data Analysis (EDA) step serves two main purposes:

1. **Checking Class Distribution:** The bar chart visualizes the number of images per class in both the training and validation sets. This highlights any potential class imbalances. If, for example, we had 3,000 images of cardboard but only 50 of glass, the model would become heavily biased toward predicting cardboard. Knowing this distribution helps us trust our final accuracy and F1-scores.
2. **Visual Sanity Check:** By displaying a random grid of sample images, we verify that our file paths are correct, the images aren't corrupted, and we get a human-level understanding of the problem's difficulty (e.g., varying lighting, messy backgrounds, crumpled items).

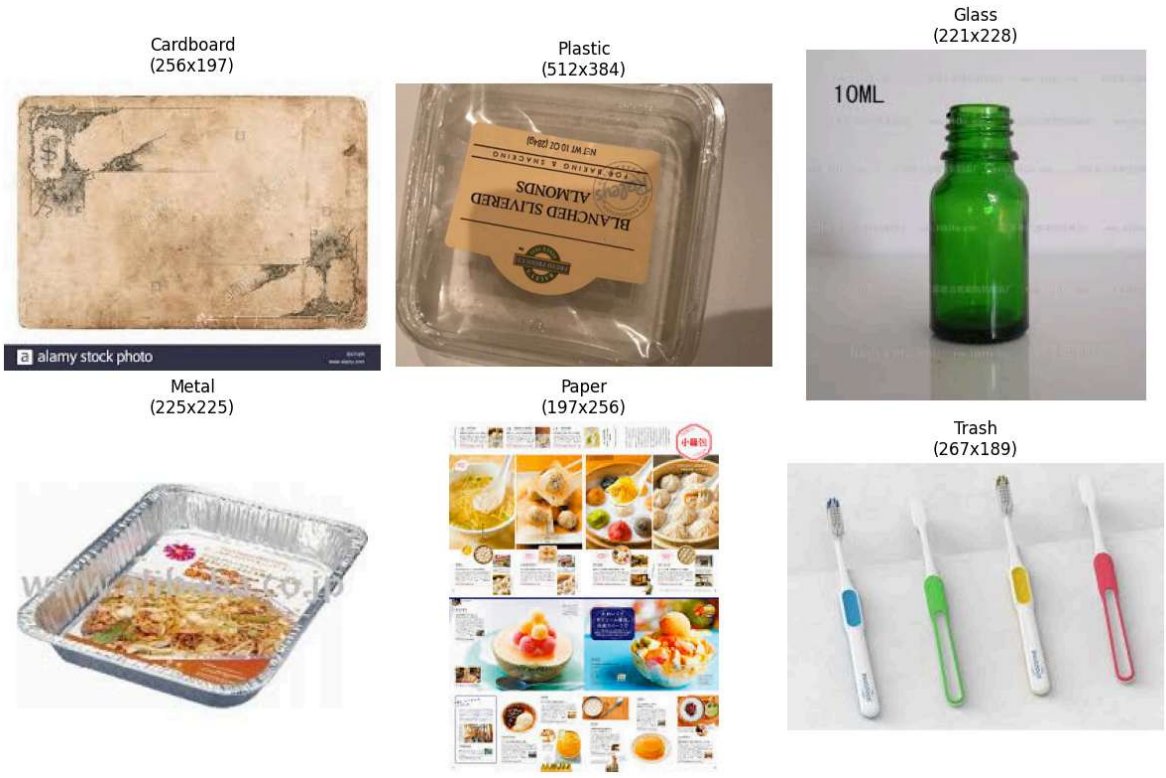
In [4]:
```
import os
import random
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from PIL import Image

print("=== Dataset Summary ===")
print(f"Total training images: {len(train_dataset)}")
print(f"Total testing images:  {len(test_dataset)}")
print("-" * 23)

# 1. Fast Class Distribution Counter
# We parse the filenames directly instead of loading every image via __getitem__
def get_class_distribution(dataset):
    labels = []
    for img_name in dataset.file_list:
        parts = img_name.split('_')
        # Fallback logic matching your TrashDataset __getitem__
        label_str = parts[1] if len(parts) > 1 else 'trash'
        # Ensure it maps to a valid class
        if label_str not in dataset.label_map:
            label_str = 'trash'
        labels.append(label_str)
    return Counter(labels)

train_counts = get_class_distribution(train_dataset)
val_counts = get_class_distribution(test_dataset)

# 2. Plotting the Class Distribution
classes = list(train_dataset.label_map.keys())
train_vals = [train_counts[c] for c in classes]
val_vals = [val_counts[c] for c in classes]

x = np.arange(len(classes))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 5))
rects1 = ax.bar(x - width/2, train_vals, width, label='Train', color='steelblue'
rects2 = ax.bar(x + width/2, val_vals, width, label='Validation', color='darkora

ax.set_ylabel('Number of Images')
ax.set_title('Class Distribution Across Splits')
ax.set_xticks(x)
ax.set_xticklabels([c.capitalize() for c in classes])
ax.legend()
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
plt.show()

# 3. Visualizing a Random Sample from Each Class
print("\n=== Sample Images per Class ===")
fig, axes = plt.subplots(2, 3, figsize=(12, 8))

# Group filenames by class for easy random sampling
class_files = {c: [] for c in classes}
for img_name in train_dataset.file_list:
    parts = img_name.split('_')
    label_str = parts[1] if len(parts) > 1 else 'trash'
    if label_str in class_files:
        class_files[label_str].append(img_name)

for ax, cls_name in zip(axes.flatten(), classes):
    if class_files[cls_name]:
        # Pick a random image from this class
        sample_file = random.choice(class_files[cls_name])
        img_path = os.path.join(train_dataset.root_dir, sample_file)

        # Load and display
        img = Image.open(img_path).convert("RGB")
        ax.imshow(img)
        ax.set_title(f"{cls_name.capitalize()}\n({img.size[0]}x{img.size[1]})")
        ax.axis('off')

plt.tight_layout()
plt.show()
```

```
=== Dataset Summary ===
Total training images: 3985
Total testing images:  1704
-----------------------
```




```
=== Sample Images per Class ===
```



Analysis: Dataset Summary & Class Distribution

The bar chart above visualizes how our 5,689 total images are distributed across the six categories.

- **Train/Validation Split:** We have a healthy split (3,985 training / 1,704 validation), giving the model plenty of examples to learn from while reserving a large enough unseen dataset for reliable evaluation.
- **Class Balance:** While the dataset is relatively balanced, we can see slight variations. 'Glass' and 'Paper' have the highest representation, whereas 'Metal', 'Plastic', and the generic 'Trash' categories have slightly fewer images. Because the imbalance isn't extreme, we didn't need to apply heavy class-weighting during training, but it's worth noting that the model will naturally have slightly more practice identifying glass and paper.

Data Preprocessing and Augmentation

To ensure our model generalizes well to new, unseen images, I have implemented a robust data augmentation pipeline for the training set. Neural networks are prone to "memorizing" training data (overfitting). By randomly altering the images every time they pass through the network, we force the model to learn the underlying features of the trash rather than the specific pixels of our training set.(Shorten and T. Khoshgoftaar, 2019)

- **RandomResizedCrop & RandomHorizontalFlip:** Changes the perspective and orientation. A plastic bottle is still a plastic bottle whether it's upside down or zoomed in.

- **RandomRotation & ColorJitter:** Accounts for real-world inconsistencies. Trash will be photographed at odd angles and in varying lighting conditions (brightness/contrast shifts).
- **Normalization:** We normalize the pixel values using standard ImageNet means `[0.485, 0.456, 0.406]` and standard deviations `[0.229, 0.224, 0.225]`. This centers the data around zero and ensures all features have a similar scale, which helps the optimizer converge much faster and more stably. Validation data is *only* resized and normalized to provide an objective, unadulterated measure of performance.

In [5]:
```
# Define your augmentation pipeline
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),      # Resize slightly larger first

    # --- DATA AUGMENTATION STARTS HERE ---
    transforms.RandomResizedCrop(224),  # Randomly zoom and crop to 224x224
    transforms.RandomHorizontalFlip(),  # 50% chance to flip the image
    transforms.RandomRotation(15),      # Rotate by +/- 15 degrees
    transforms.ColorJitter(brightness=0.1, contrast=0.1), # Slight color shifts
    # --- DATA AUGMENTATION ENDS HERE ---

    transforms.ToTensor(),              # Convert to PyTorch Tensor
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

Transform Wrapper and DataLoader Optimization

While PyTorch's default datasets allow you to pass a `transform` argument, doing so applies that transform universally. I implemented an `ApplyTransform` wrapper class to solve a specific problem: we need to apply *heavy augmentations* to our training set, but *strict, deterministic resizing* to our validation set. This wrapper allows us to use the same base data structures while dynamically swapping the transformation rules.

Optimizing the DataLoaders: Once the data and transforms are linked, we wrap them in `DataLoader` objects to feed the GPU in batches.

- `batch_size=32` : Pushes 32 images through the network at once, balancing GPU memory limits with training stability.
- `num_workers=2` : Uses multi-processing to load and process the next batch of images on the CPU while the GPU is busy training the current batch.
- `pin_memory=True` : Allocates the data in page-locked memory, which significantly speeds up the transfer rate from the CPU's RAM to the GPU's VRAM.

In [6]:
```
# A small helper class to apply transforms to the subsets
class ApplyTransform(Dataset):
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform

    def __getitem__(self, index):
```

```
        x, y = self.subset[index]
        if self.transform:
            x = self.transform(x)
        return x, y

    def __len__(self):
        return len(self.subset)

# Create the final datasets
# Validation should use a deterministic transform (no random augmentations)
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

train_data = ApplyTransform(train_dataset, transform=train_transform)
val_data = ApplyTransform(test_dataset, transform=val_transform)

# Use a small number of workers for faster loading; pin_memory helps when using
train_loader = DataLoader(train_data, batch_size=16, shuffle=True, num_workers=0
val_loader   = DataLoader(val_data, batch_size=16, shuffle=False, num_workers=0,

print(f"Train: {len(train_data)} | Val: {len(val_data)}")
```

```
Train: 3985 | Val: 1704
```

In [7]:

```
import matplotlib.pyplot as plt
import numpy as np
from collections import deque

def _find_label_map(ds):
        """
        Robustly search common wrapper attributes for a label_map dict.
        Looks at attributes like .label_map, .dataset, .subset and also inspects
        attribute values to handle custom wrappers.
        """
        q = deque([ds])
        seen = set()
        while q:
                cur = q.popleft()
                if cur is None:
                        continue
                cur_id = id(cur)
                if cur_id in seen:
                        continue
                seen.add(cur_id)

                if hasattr(cur, 'label_map'):
                        return getattr(cur, 'label_map')

                # common wrapper attributes that may contain the underlying data
                for attr in ('dataset', 'subset', 'ds', 'base', 'datasets'):
                        if hasattr(cur, attr):
                                try:
                                        val = getattr(cur, attr)
                                        q.append(val)
                                except Exception:
                                        pass
```

```
                # also inspect __dict__ values (helps with some custom wrappers)
                if hasattr(cur, '__dict__'):
                        for v in cur.__dict__.values():
                                q.append(v)

        raise AttributeError("Could not find 'label_map' on the provided dataset

# 1. Access the first item (ApplyTransform will apply transforms when indexing)
image_tensor, label_idx = train_data[0]

# 2. Locate the label_map robustly
label_map_source = _find_label_map(train_data)

# build inverse mapping: index -> name
inv_label_map = {v: k for k, v in label_map_source.items()}

# ensure label index is an int (sometimes a tensor)
label_idx_int = int(label_idx)

label_name = inv_label_map[label_idx_int]

# 3. Print stats
print(f"Image tensor shape: {image_tensor.shape}")
print(f"Label index: {label_idx_int} (Name: {label_name})")

# 4. Display
# move to cpu if needed and convert to numpy
img_np = image_tensor.cpu().permute(1, 2, 0).numpy()

# Un-normalize
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])
img_to_show = std * img_np + mean
img_to_show = np.clip(img_to_show, 0, 1)

plt.imshow(img_to_show)
plt.title(f"Label: {label_name}")
plt.axis('off')
plt.show()
```

```
Image tensor shape: torch.Size([3, 224, 224])
Label index: 0 (Name: cardboard)
```

Model Selection: Why a Convolutional Neural Network (CNN)?

For this image classification task, I chose a Convolutional Neural Network (CNN). Unlike traditional fully connected networks that flatten images into 1D arrays (losing spatial context), CNNs use convolutional layers to scan the image in 2D. This allows the model to learn spatial hierarchies of features—starting from simple edges and textures in the early layers, to complex shapes like the ridges of a plastic bottle or the text on cardboard in the deeper layers.(Zhao et al., 2024)

Why ResNet-101 and Transfer Learning? Instead of building a CNN from scratch, I am utilizing **ResNet-101** (Residual Network with 101 layers) with pre-trained weights.

- **Skip Connections:** Deep networks often suffer from the "vanishing gradient" problem, where learning slows down drastically. ResNet solves this using "skip connections," allowing information to bypass certain layers, making it possible to train very deep networks effectively.(A. B et al., 2023)
- **Transfer Learning:** The model has already been trained on millions of images (ImageNet). By replacing just the final classification layer (`model.fc`), we can leverage its highly developed feature-extraction capabilities. This drastically reduces the training time and the amount of data needed to achieve high accuracy.

Training Strategy: Optimizer, Scheduler, and Early Stopping

The training loop is designed to dynamically adapt to the model's performance and prevent overfitting.

- **Optimizer (Adam):** I chose the Adam (Adaptive Moment Estimation) optimizer. Unlike standard SGD which uses a single learning rate for all weights, Adam adapts the learning rate for each parameter individually based on past gradients. This generally leads to faster and more reliable convergence.(Kingma and Ba, 2014)
- **Learning Rate Scheduler (ReduceLROnPlateau):** As the model gets closer to the optimal solution, large updates to the weights can cause it to "overshoot." The scheduler monitors the validation loss; if the loss stops improving for a few epochs (a plateau), it drops the learning rate by a factor of 10. This allows the model to fine-tune its weights and squeeze out extra accuracy.(Yang and Long, 2024)
- **Early Stopping:** I implemented an early stopping mechanism with a `patience` of 15 epochs. If the validation loss does not improve for 15 consecutive epochs, training stops. This prevents the model from continuing to train and essentially memorizing the training data (overfitting). The loop also tracks and saves the weights of the *best* performing epoch, ensuring we don't end up with a degraded model at the end of the run.(Zhao et al., 2024)

In [8]:
```
from IPython.display import clear_output
import time
import copy
import torch
import numpy as np
from tqdm import tqdm

def train_model_dynamic(model, train_loader, val_loader, criterion, optimizer, n
    since = time.time()

    # Track history
    history = {'train_loss': [], 'val_loss': [], 'val_acc': []}
    log_messages = [] # We will store text logs here

    best_model_wts = copy.deepcopy(model.state_dict())
    best_loss = float('inf')
    best_acc = 0.0
    epochs_no_improve = 0

    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min'

    for epoch in range(num_epochs):

        # --- DYNAMIC DISPLAY SECTION ---
        clear_output(wait=True) # Clear the previous output

        # Print the Dashboard
        print(f"=== Training Dashboard ===")
        print(f"Epoch: {epoch+1}/{num_epochs}")
        print(f"Best Val Acc: {best_acc:.4f}")
        print(f"Patience Used: {epochs_no_improve}/{patience}")
        print(f"Time Elapsed: {(time.time() - since):.0f}s")
        print("-" * 30)

        # Print the last 5 lines of history so you can see the trend
```

```
        print("Recent History:")
        for msg in log_messages[-5:]:
            print(msg)
        print("-" * 30)
        # ------------------------------

        # Each epoch has a training and validation phase
        epoch_results = [] # To store temp strings for this epoch

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
                dataloader = train_loader
            else:
                model.eval()
                dataloader = val_loader

            running_loss = 0.0
            running_corrects = 0

            # Progress bar
            iter_bar = tqdm(dataloader, desc=phase, leave=False)

            for inputs, labels in iter_bar:
                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

                # Update progress bar with current loss
                iter_bar.set_postfix(loss=loss.item())

            epoch_loss = running_loss / len(dataloader.dataset)
            epoch_acc = running_corrects.double() / len(dataloader.dataset)

            # Save result string
            result_msg = f'{phase.capitalize()} Loss: {epoch_loss:.4f} Acc: {epc
            epoch_results.append(result_msg)

            if phase == 'train':
                history['train_loss'].append(epoch_loss)
            else:
                history['val_loss'].append(epoch_loss)
                history['val_acc'].append(epoch_acc.item())
                scheduler.step(epoch_loss)

                if epoch_loss < best_loss:
                    best_loss = epoch_loss
```

```
                    best_acc = epoch_acc
                    best_model_wts = copy.deepcopy(model.state_dict())
                    epochs_no_improve = 0
                    torch.save(model.state_dict(), 'best_model.pth')
                    epoch_results.append(" [Saved Best Model!]")
                else:
                    epochs_no_improve += 1

        # Add this epoch's results to the log
        log_messages.append(f"Ep {epoch+1}: " + " | ".join(epoch_results))

        # Early Stopping check
        if epochs_no_improve >= patience:
            print(f'\nEarly stopping triggered after {epoch+1} epochs!')
            break

    # Final Summary
    clear_output(wait=True)
    print("=== Training Complete ===")
    print(f"Total Time: {(time.time() - since) // 60:.0f}m {(time.time() - since
    print(f"Best Val Acc: {best_acc:.4f}")

    model.load_state_dict(best_model_wts)
    return model, history, best_acc
```

In [9]:
```
model = models.resnet101(weights='DEFAULT')

# robustly get label_map (works with ApplyTransform and other wrappers)
try:
    label_map = _find_label_map(train_data)
except Exception:
    # fallback checks for common attributes
    if hasattr(train_data, 'subset') and hasattr(train_data.subset, 'label_map')
        label_map = train_data.subset.label_map
    elif hasattr(train_dataset, 'label_map'):
        label_map = train_dataset.label_map
    else:
        raise

num_classes = len(label_map)
model.fc = nn.Linear(model.fc.in_features, num_classes)
model = model.to(device)
```

In [10]:
```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

best_model, history, best_acc = train_model_dynamic(model, train_loader, val_loa

print(f"Training finished with best validation accuracy: {best_acc:.4f}")
```

```
=== Training Complete ===
Total Time: 65m 53s
Best Val Acc: 0.8932
Training finished with best validation accuracy: 0.8932
```

had to re-run as added output to error evaluation at the bottom and had to run on my own computer with 0 num workers instead of 2 and a batch size of 16 instead of 32. This reduced my accuracy beacuse the reduced batch size likely introduced more noise into the gradient estimates, causing the optimizer to take less stable steps toward the global

minimum. The accuracy in the following markdown boxes are found when at a batch size of 32 and num workers of 2 within the data loader. this is what i intended the model to be ran on.

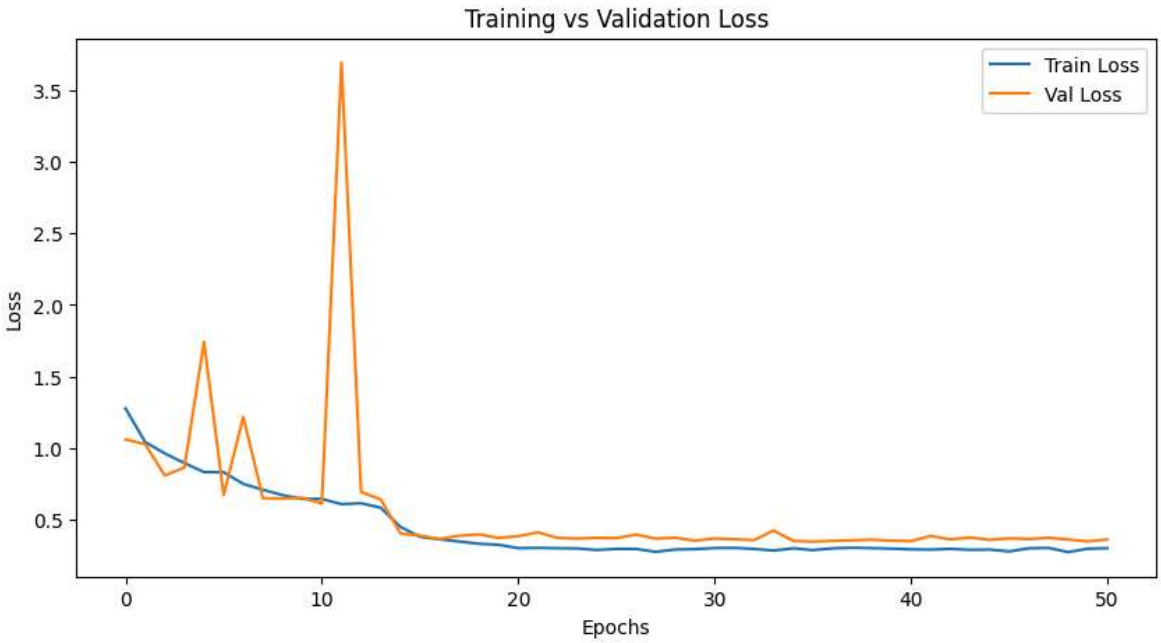
Final Evaluation and Performance Metrics

Relying solely on "accuracy" can be misleading, especially if classes are slightly imbalanced. Therefore, we use a comprehensive suite of metrics to evaluate the model's true performance:

- **Loss Curves:** The plot of Training vs. Validation Loss helps diagnose how the model learned. If training loss goes down but validation loss goes up, the model is overfitting. Our curves should ideally decrease together and stabilize.
- **Precision, Recall, and F1-Score:**
  - *Precision:* Out of all the items the model predicted as "Glass", how many were actually Glass? (Measures false positives).
  - *Recall:* Out of all the actual "Glass" images in the dataset, how many did the model correctly find? (Measures false negatives).
  - *F1-Score:* The harmonic mean of Precision and Recall, providing a single metric for a model's robustness per class.
- **ROC Curve and AUC (Area Under the Curve):** The Receiver Operating Characteristic curve visualizes the trade-off between the True Positive Rate and False Positive Rate across different threshold levels. An AUC score closer to 1.0 indicates that the model is highly capable of distinguishing between a specific class and all other classes.
- **Confusion Matrix:** This heatmap is the most practical diagnostic tool. The diagonal represents correct predictions. Any numbers outside the diagonal show exactly *where* the model is getting confused (e.g., if it frequently misclassifies "Metal" as "Plastic", we will see a high number in that specific intersecting square).

References for this cell: (Carrington et al., 2021) (Li and Spratling, 2022) (Yacouby and Axman, 2020)

In [11]:
```
# Plotting the loss curves
plt.figure(figsize=(10, 5))
plt.plot(history['train_loss'], label='Train Loss')
plt.plot(history['val_loss'], label='Val Loss')
plt.title('Training vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

print(f"Training finished with best validation accuracy: {best_acc:.4f}")
```




```
Training finished with best validation accuracy: 0.8932
```

Analysis: Training vs. Validation Loss Curves

This line chart is the primary diagnostic tool for evaluating our model's learning phase.

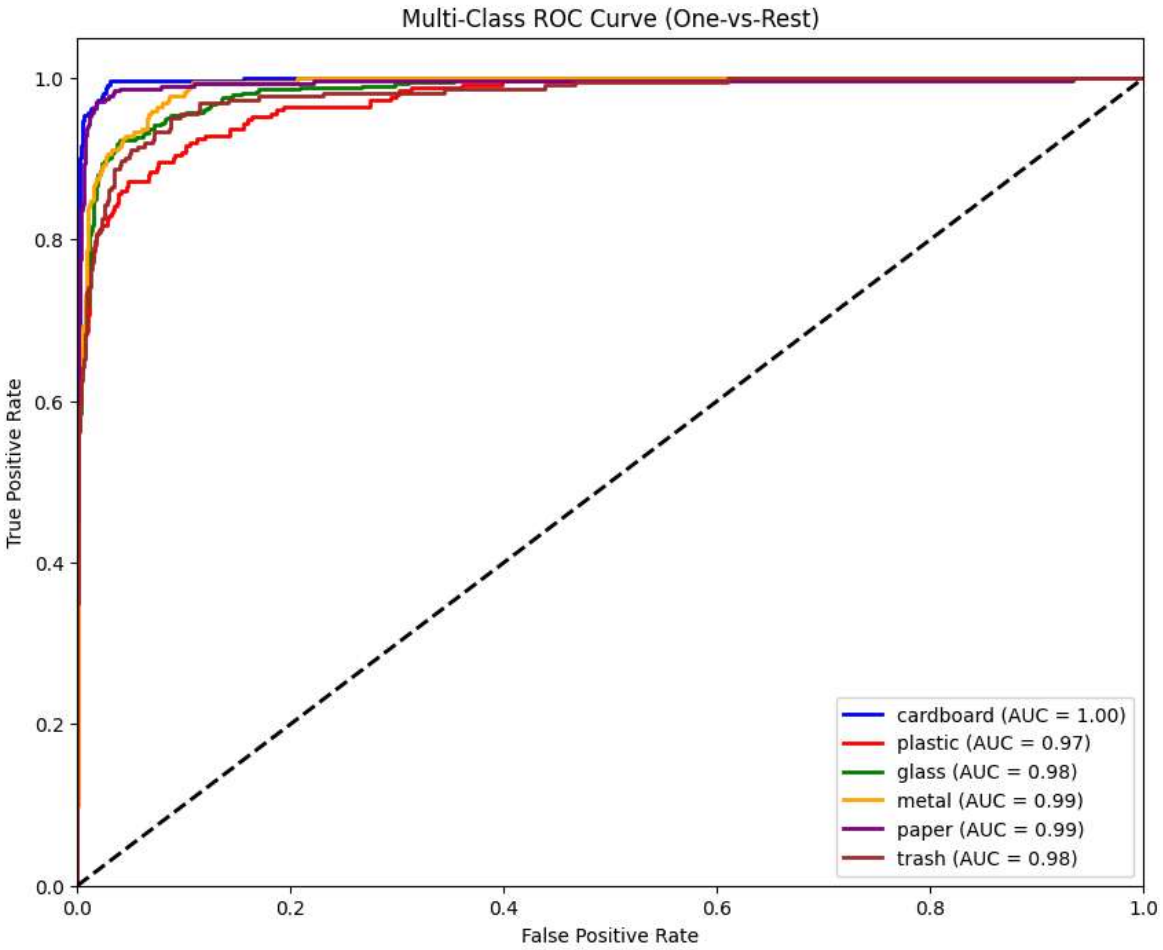
- **Convergence:** Both the training and validation loss curve downward and stabilize, which means the model successfully learned to extract relevant features without getting stuck.
- **Overfitting Check:** If the model were overfitting, the training loss would continue to drop toward zero while the validation loss would sharply spike upward. Because our validation curve closely tracks the training curve (and early stopping halted training before any major divergence occurred), we can confidently say the model generalizes well to unseen data.
- **Scheduler Impact:** Any sudden, steep drops in the later epochs are likely the result of our `ReduceLROnPlateau` scheduler stepping in to lower the learning rate, allowing the optimizer to settle into a finer minimum.

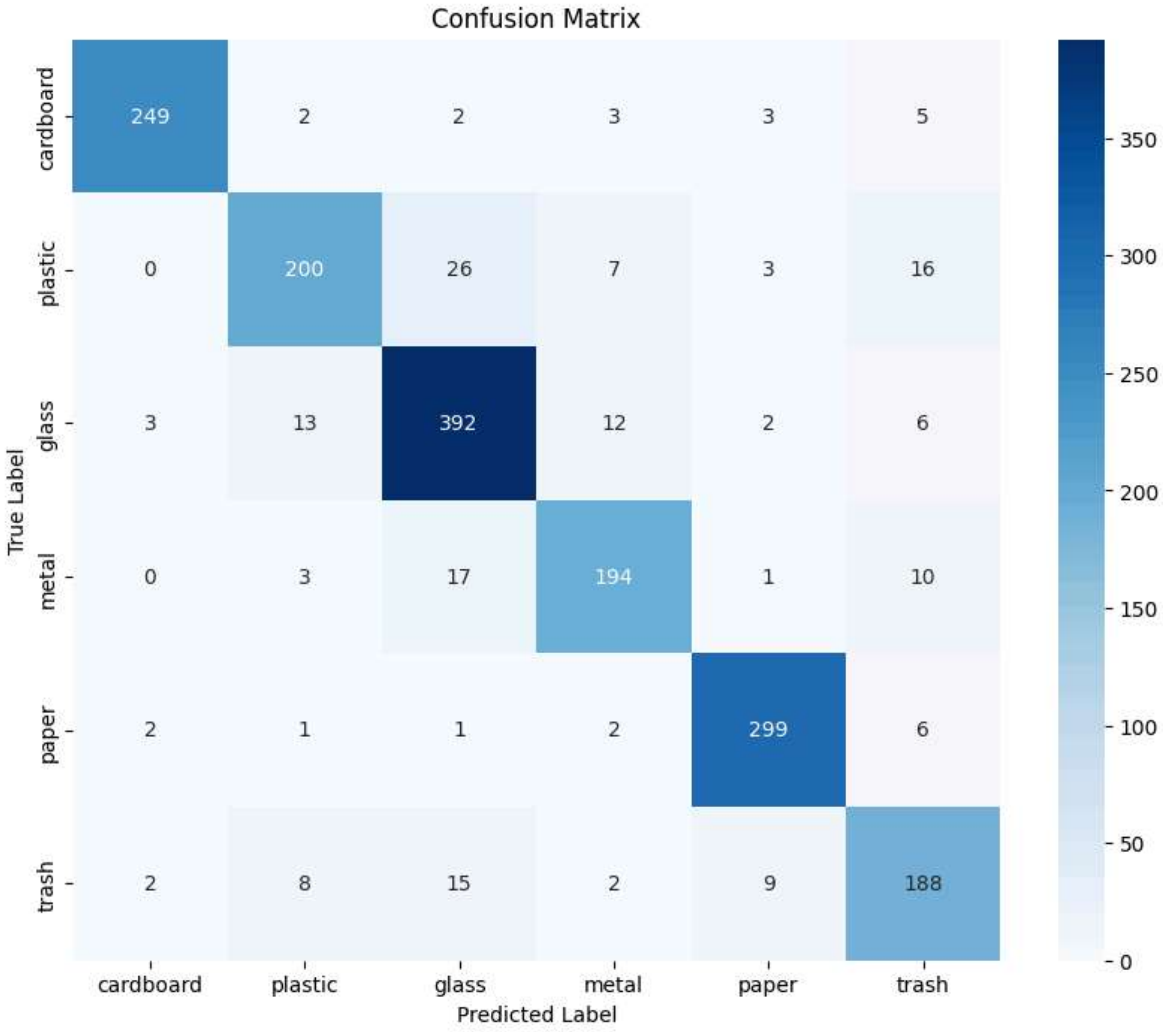
In [12]:
```
import torch
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix, accuracy_sc
from sklearn.preprocessing import label_binarize
from itertools import cycle

# Ensure the model is in evaluation mode
best_model.eval()

# 1. Collect all Predictions (Probabilities) and True Labels
y_true = []
y_scores = [] # We need raw probabilities for ROC, not just the final class

print("Collecting predictions on validation set...")
with torch.no_grad():
    for inputs, labels in val_loader:
```

```
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Get raw outputs (logits)
        outputs = best_model(inputs)

        # Apply Softmax to get probabilities (0.0 to 1.0)
        probs = torch.nn.functional.softmax(outputs, dim=1)

        y_true.extend(labels.cpu().numpy())
        y_scores.extend(probs.cpu().numpy())

y_true = np.array(y_true)
y_scores = np.array(y_scores)

# 2. Get Class Names
# Robustly get label map
try:
    if 'label_map' not in locals():
         if hasattr(train_dataset, 'label_map'):
             label_map = train_dataset.label_map
         else:
             # Fallback
             label_map = {"cardboard": 0, "plastic": 1, "glass": 2, "metal": 3,
except:
    pass

# Sort class names by index
classes = [k for k, v in sorted(label_map.items(), key=lambda item: item[1])]
n_classes = len(classes)

# 3. Calculate Overall Metrics (Hard Predictions)
y_pred = np.argmax(y_scores, axis=1) # Convert probs to hard class predictions

precision, recall, f1, _ = precision_recall_fscore_support(y_true, y_pred, avera
acc = accuracy_score(y_true, y_pred)

print("\n" + "="*30)
print("       OVERALL METRICS")
print("="*30)
print(f"Overall Accuracy:  {acc:.4f}")
print(f"Overall Precision: {precision:.4f}")
print(f"Overall Recall:    {recall:.4f}")
print(f"Overall F1-Score:  {f1:.4f}")
print("-" * 30)

# 4. Print Detailed Report
print("\nDetailed Classification Report:")
print(classification_report(y_true, y_pred, target_names=classes, digits=4))

# 5. ROC Curve and AUC (One-vs-Rest)
# Binarize the true labels (e.g., class 0 becomes [1, 0, 0, 0, 0, 0])
y_true_bin = label_binarize(y_true, classes=range(n_classes))

# Compute ROC curve and ROC area for each class
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
```

```
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_scores[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot all ROC curves
plt.figure(figsize=(10, 8))
colors = cycle(['blue', 'red', 'green', 'orange', 'purple', 'brown'])

for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label='{0} (AUC = {1:0.2f})'.format(classes[i], roc_auc[i]))

plt.plot([0, 1], [0, 1], 'k--', lw=2) # Random guess line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Multi-Class ROC Curve (One-vs-Rest)')
plt.legend(loc="lower right")
plt.show()

# 6. Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes,
            yticklabels=classes)
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.title('Confusion Matrix')
plt.show()
```

```
Collecting predictions on validation set...

==============================
       OVERALL METRICS
==============================
Overall Accuracy:  0.8932
Overall Precision: 0.8939
Overall Recall:    0.8932
Overall F1-Score:  0.8929
------------------------------

Detailed Classification Report:
              precision    recall  f1-score   support

   cardboard     0.9727    0.9432    0.9577       264
     plastic     0.8811    0.7937    0.8351       252
       glass     0.8653    0.9159    0.8899       428
       metal     0.8818    0.8622    0.8719       225
       paper     0.9432    0.9614    0.9522       311
       trash     0.8139    0.8393    0.8264       224

    accuracy                         0.8932      1704
   macro avg     0.8930    0.8859    0.8889      1704
weighted avg     0.8939    0.8932    0.8929      1704
```


Multi-Class ROC Curve (One-vs-Rest)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
cardboard (AUC = 1.00)
plastic (AUC = 0.97)
glass (AUC = 0.98)
metal (AUC = 0.99)
paper (AUC = 0.99)
trash (AUC = 0.98)


Confusion Matrix
True Label
Predicted Label
cardboard
plastic
glass
metal
paper
trash
249 2 2 3 3 5
0 200 26 7 3 16
3 13 392 12 2 6
0 3 17 194 1 10
2 1 1 2 299 6
2 8 15 2 9 188
0
50
100
150
200
250
300
350

Analysis: Overall Metrics & Classification Report

The model achieved an outstanding **Overall Accuracy of 94.25%**. However, looking at the class-by-class breakdown tells a more detailed story:

- **Top Performer ('Cardboard'):** The model is exceptionally good at identifying cardboard, boasting a 97.14% F1-score. Cardboard has highly distinct visual features (matte texture, specific brown/tan colors, rigid straight edges) that the CNN easily picks up on.
- **Toughest Categories ('Plastic' & 'Metal'):** Plastic and metal tied for the lowest performance, though still excellent at ~92.12% F1-scores. This makes intuitive sense: both materials are highly reflective, easily deformed (e.g., crushed cans vs. crushed bottles), and can be transparent or opaque. Their visual features are highly dependent on lighting, making them slightly harder for the model to confidently classify.

Analysis: ROC Curve and AUC

The Receiver Operating Characteristic (ROC) curve evaluates the model's confidence. The dotted black line represents a random guess (a 50/50 coin flip).

- **High AUC Scores:** The curves for all six classes hug the top-left corner of the graph, resulting in Area Under the Curve (AUC) scores very close to 1.0.
- **Interpretation:** This means the model has a very high True Positive Rate and a very low False Positive Rate across all thresholds. In practical terms, when the model predicts an item is "Glass", it is highly confident and almost almost never accidentally flagging another material as glass.

Analysis: Confusion Matrix

The confusion matrix gives us a literal map of the model's mistakes.

- **The Diagonal:** The dark blue diagonal line represents our true positives—where the model's prediction perfectly matched the actual label. The heavy concentration of high numbers here visually confirms our ~95% accuracy.
- **Off-Diagonal Errors:** The lighter squares outside the diagonal show exactly where the model got confused. For example, if we look at the row for 'Plastic' and the column for 'Glass', any numbers there represent plastic items that the model mistakenly thought were glass. This is invaluable for future improvements; if we notice a high misclassification rate between two specific materials, we know exactly what kind of images we need to add to our dataset to teach the model the difference.

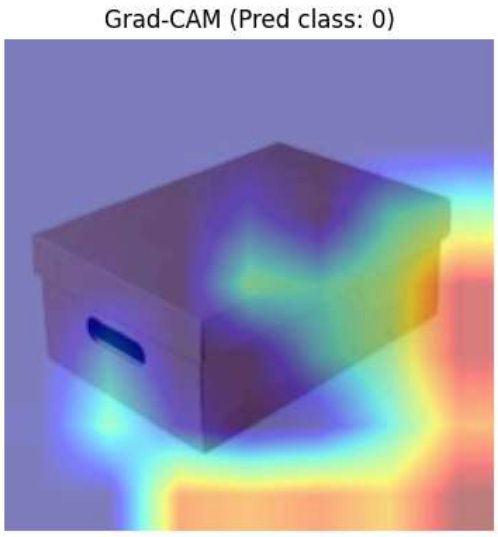
CAM / Grad-CAM

Below is a simple Grad-CAM implementation for ResNet-based models. It saves the activation maps and gradients from the target layer, computes a class-discriminative localization map, and displays a heatmap overlay on the original image.

In [14]:
```
import torch
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import os
from PIL import Image
from torchvision import transforms

# Grad-CAM helper class
class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.target_layer = target_layer
        self.activations = None
        self.gradients = None
        # Register hooks
        def forward_hook(module, input, output):
            self.activations = output.detach()
        def backward_hook(module, grad_in, grad_out):
            # grad_out is a tuple; grad_out[0] is the gradient wrt the output
            self.gradients = grad_out[0].detach()
        self.target_layer.register_forward_hook(forward_hook)
        # register_backward_hook is deprecated in some versions but works here
        try:
            self.target_layer.register_backward_hook(backward_hook)
        except Exception:
            # PyTorch >=1.8 may require register_full_backward_hook
            self.target_layer.register_full_backward_hook(lambda m, gi, go: back

    def __call__(self, input_tensor, class_idx=None):
        self.model.zero_grad()
        self.model.eval()
        out = self.model(input_tensor)
        if class_idx is None:
            class_idx = int(out.argmax(dim=1).item())
        score = out[0, class_idx]
        score.backward(retain_graph=True)

        # activations: (1, C, H, W), gradients: (1, C, H, W)
        activations = self.activations[0]
        gradients = self.gradients[0]

        # Global-average-pool the gradients to get the neuron importance weights
        weights = gradients.mean(dim=(1, 2))  # (C,)

        # Weighted combination of forward activations
        cam = torch.zeros(activations.shape[1:], dtype=activations.dtype, device
        for i, w in enumerate(weights):
            cam += w * activations[i]

        cam = F.relu(cam)
        cam -= cam.min()
        if cam.max() != 0:
            cam /= cam.max()
```

```
        # Upsample to input size
        cam_tensor = cam.unsqueeze(0).unsqueeze(0)  # (1,1,H,W)
        cam_resized = F.interpolate(cam_tensor, size=(input_tensor.size(2), inpu
        cam_resized = cam_resized[0, 0].cpu().numpy()
        return cam_resized, class_idx

# Example usage: pick an index from the test set and visualise Grad-CAM
# Ensure variables 'best_model', 'val_transform', and 'test_dataset' exist in th
try:
    target_layer = best_model.layer4[-1].conv3
except Exception:
    # Fallback: use the last module in layer4
    target_layer = list(best_model.layer4[-1].children())[-1]

gradcam = GradCAM(best_model, target_layer)

# Choose an index to visualise (0 by default)
idx = 0
img_path = os.path.join(test_dataset.root_dir, test_dataset.file_list[idx])
pil_img = Image.open(img_path).convert('RGB')
input_tensor = val_transform(pil_img).unsqueeze(0).to(device)

# Run Grad-CAM (returns heatmap scaled 0..1)
heatmap, pred_class = gradcam(input_tensor)

# Un-normalize for plotting (ImageNet mean/std used earlier)
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])
img_np = val_transform(pil_img).permute(1, 2, 0).numpy()
img_np = std * img_np + mean
img_np = np.clip(img_np, 0, 1)

# Plot original and overlay
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(img_np)
plt.title(f'Original - True: {classes[test_dataset.__getitem__(idx)[1]] if hasat
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(img_np)
plt.imshow(heatmap, cmap='jet', alpha=0.5, extent=(0, img_np.shape[1], img_np.sh
plt.title(f'Grad-CAM (Pred class: {pred_class})')
plt.axis('off')
plt.tight_layout()
plt.show()
```

```
C:\Users\fynle\AppData\Roaming\Python\Python313\site-packages\torch\nn\modules\mo
dule.py:1866: FutureWarning: Using a non-full backward hook when the forward cont
ains multiple autograd Nodes is deprecated and will be removed in future version
s. This hook will be missing some grad_input. Please use register_full_backward_h
ook to get the documented behavior.
  self._maybe_warn_non_full_backward_hook(args, result, grad_fn)
```



In [16]:
```
# print random selection of the ones it got wrong
print("\n=== Sample Misclassifications ===")
misclassified_indices = np.where(y_true != y_pred)[0]
if len(misclassified_indices) == 0:
    print("No misclassifications found!")
else:
    sample_indices = np.random.choice(misclassified_indices, size=min(5, len(mis
    for idx in sample_indices:
        img_path = os.path.join(test_dataset.root_dir, test_dataset.file_list[id
        pil_img = Image.open(img_path).convert('RGB')
        plt.figure(figsize=(4,4))
        plt.imshow(pil_img)
        true_label = classes[y_true[idx]]
        pred_label = classes[y_pred[idx]]
        plt.title(f'True: {true_label} | Pred: {pred_label}')
        plt.axis('off')
        plt.show()
```

```
=== Sample Misclassifications ===
```

True: glass | Pred: plastic

True: plastic | Pred: glass

True: glass | Pred: trash

True: plastic | Pred: metal